

Оптимизированное матричное умножение

Владимир Попов

20 апреля 2026 г.

Содержание

1	Характеристики	3
1.1	Бенчмаркинг	3
2	Реализация генератора minstd_{rand}	3
2.1	Скалярная версия	4
2.2	Векторная версия	5
3	Бенчмаркинг	7
3.1	latency и throughput	7
3.2	Вычисление числа π	7

1 Характеристики

Параметр	Значение
<i>Аппаратное обеспечение (Hardware)</i>	
Процессор (CPU)	AMD Ryzen 7 8845H (8 ядер, 16 потоков)
Архитектура	x86_64
Базовая / Максимальная частота	3.8 ГГц / 5.1 ГГц
L1 Кэш (данные/инструкции)	256 / 256 КиБ
L2 Кэш	8 МиБ
L3 Кэш	16 МиБ (общий)
Оперативная память	32 ГБ LPDDR5x
<i>Программное обеспечение (Software)</i>	
Операционная система	Ubuntu 22.04.5 LTS
Ядро Linux	6.8.0-106-generic
Компилятор	GCC 11.4 / Clang 11.4
Библиотека C	glibc 2.35
Флаги компиляции	-DNDEBUG -O3 -march=native

Таблица 1: Характеристики тестовой системы и программного окружения

На моей машине есть поддержка следующих расширений:

AES, AMD-V AVX, AVX2, AVX512, FMA3, MMX-plus, SHA, SSE, SSE2, SSE3, SSE4.1, SSE4.2, SSE4A, SSSE3.

Также имеется 2 исполнительных FMA блока.

1.1 Бенчмаркинг

Бенчмаркинг производится при помощи библиотеки написанной ранее в задаче 11. Её описание и функционал можете прочитать в README, пункт 11.

2 Реализация генератора `minstdrand`

Эталоном считается `std::minstd_rand`, который возвращает случайные числа в диапазоне $[1, 2^{31} - 2]$ по формуле

$$x_{i+1} = (a \cdot x_i + c) \mod m \quad (1)$$

, где $a = 48271$, $c = 0$, $m = 2^{31} - 1$, $x_0 = seed$

2.1 Скалярная версия

m - число Мерсена, то есть

$$2^{31} \equiv 1 \pmod{2^{31}-1} \quad (2)$$

В нашей формуле мы перемножаем a и x_i , и результат укладывается в 64 бита. Разобьём результат $z = a \cdot x_i$ на нижние 31 бит и на верхние 33.

$$\begin{aligned} x_{i+1} &= (2^{31} \cdot z_{hi} + z_{lo}) \pmod{2^{31} - 1} = \\ &= (2^{31} \cdot z_{hi}) \pmod{2^{31} - 1} + z_{lo} \pmod{2^{31} - 1} \stackrel{(1)}{=} \\ &\stackrel{(1)}{=} (z_{hi} + z_{lo}) \pmod{2^{31} - 1} \end{aligned} \quad (3)$$

, где $z_{hi} = z[63 : 31] = z \gg 31$, $z_{lo} = z[30 : 0] = z \& 0x7FFFFFFF$

Таким образом возьмём модуль от произведения, а потом и от получившейся суммы.

```

1 static inline constexpr const uint32_t MULTIPLIER_ = UINT32_C(48271);
2 static inline constexpr const uint32_t MODULUS_POW_ = 31;
3 static inline constexpr const uint32_t MODULUS_ = ((UINT32_C(1) << MODULUS_POW_) - 1)
4
5 uint32_t MinstdRand::operator()() noexcept {
6     const uint64_t product = (uint64_t)MULTIPLIER_ * prev_val_;
7     const uint32_t lo = (uint32_t)product & MODULUS_;
8     const uint32_t hi = (uint32_t)(product >> MODULUS_POW_);
9     const uint32_t sum = lo + hi;
10
11     const uint32_t sum_lo = sum & MODULUS_;
12     const uint32_t sum_hi = sum >> MODULUS_POW_;
13
14     prev_val_ = sum_hi + sum_lo;
15
16     return prev_val_;
17 }
```

2.2 Векторная версия

Выразим формулу для векторной версии

- $x_{i+1} = (a \cdot x_i) \bmod m$
- $x_{i+2} = (a \cdot x_{i+1}) \bmod m = (a \cdot ((a \cdot x_i) \bmod m)) \bmod m \stackrel{A_2=a^2}{=} (A_2 \cdot x_i) \bmod m$
- $(x_{i+1}, \dots, x_{i+k}) = (A_k \cdot (x_{i+1-k}, \dots, x_i)) \bmod m$, где $A_k = a^k \bmod m$

Есть нюанс, что векторное умножение работает так, что перемножает только чётные значения операндов, так как результат может занимать в 2 раза больше битов. Поэтому отдельно будем работать с числами на чётных позициях, а отдельно - на нечётных.

```
1 static inline constexpr const uint32_t MULTIPLIER_ = UINT32_C(1098894339);
2 static inline constexpr const uint32_t MODULUS_POW_ = 31;
3 static inline constexpr const uint32_t MODULUS_ = ((UINT32_C(1) << MODULUS_POW_) - 1)
4 ;
5 static inline const __m512i multipliers_vec_ = _mm512_set1_epi32(MULTIPLIER_);
6 static inline const __m512i modulus_vec_ = _mm512_set1_epi64(MODULUS_);
7 static inline const __m512i inv_modulus_vec_ = _mm512_set1_ps(INV_MODULUS_);
8
9 __m512i MinstdRandVec::operator()() {
10     const __m512i prod_even = _mm512_mul_epu32(prev_vals_, multipliers_vec_);
11     const __m512i prev_vals_shifted = _mm512_srli_epi64(prev_vals_, 32);
12     const __m512i prod_odd = _mm512_mul_epu32(prev_vals_shifted, multipliers_vec_);
13
14     const __m512i lo_even = _mm512_and_si512(prod_even, modulus_vec_);
15     const __m512i hi_even = _mm512_srli_epi64(prod_even, MODULUS_POW_);
16     const __m512i sum_even = _mm512_add_epi64(lo_even, hi_even);
17
18     const __m512i lo_odd = _mm512_and_si512(prod_odd, modulus_vec_);
19     const __m512i hi_odd = _mm512_srli_epi64(prod_odd, MODULUS_POW_);
20     const __m512i sum_odd = _mm512_add_epi64(lo_odd, hi_odd);
21
22     const __m512i sum_lo_even = _mm512_and_si512(sum_even, modulus_vec_);
23     const __m512i sum_hi_even = _mm512_srli_epi64(sum_even, MODULUS_POW_);
24     const __m512i res_even = _mm512_add_epi64(sum_lo_even, sum_hi_even);
25
26     const __m512i sum_lo_odd = _mm512_and_si512(sum_odd, modulus_vec_);
```

```

27  const __m512i sum_hi_odd = _mm512_srli_epi64(sum_odd, MODULUS_POW_);
28  const __m512i res_odd = _mm512_add_epi64(sum_lo_odd, sum_hi_odd);
29
30  const __m512i res_odd_shifted = _mm512_slli_epi64(res_odd, 32);
31
32  const __m512i res = prev_vals_;
33
34  prev_vals_ = _mm512_or_si512(res_even, res_odd_shifted);
35
36  return res;
37 }

```

Также есть вопрос по поводу того, как инициализировать векторный генератор. Было принято решение написать следующие конструкторы, базирующиеся на скалярном генераторе, чтобы пользователю не приходилось генерировать первую последовательность.

```

1  static inline constexpr const uint64_t ALIGNMENT_ = 64;
2  static inline constexpr const size_t VALS_IN_VEC_ = 16;
3  alignas(ALIGNMENT_) static inline constexpr const uint32_t INIT_VALS_[VALS_IN_VEC_] =
4      {
5          48271,
6          /**/
7          1098894339,
8      };
9  MinstdRandVec::MinstdRandVec()
10     : seed(INIT_VALS_)
11 {}
12
13 MinstdRandVec::MinstdRandVec(uint32_t seed)
14 {
15     alignas(ALIGNMENT_) uint32_t arr[VALS_IN_VEC_];
16     MinstdRand gen(seed);
17     for (size_t ind = 0; ind < VALS_IN_VEC_; ++ind) {
18         arr[ind] = gen();
19     }
20     prev_vals_ = _mm512_load_si512(arr);
21 }

```

3 Бенчмаркинг

3.1 latency и throughput

С использованием ранее написанной библиотеки замерим latency и throughput написанных функций и эталонной из libc.

Реализация	Latency (clks)	Throughput (clks)
std::minstd_rand	114	17.895 ± 0.0034
My MinstdRand	114	8.947 ± 0.0013
My MinstdRandVec	7.125	2.2555 ± 0.00057

Условия замера:

Latency: buckets_cnt = 10, iterations_cnt = 500 000

Throughput: buckets_cnt = 10, batches_cnt = 50, iterations_cnt = 500 000

Таблица 2: Результаты бенчмарков производительности

3.2 Вычисление числа пи

Будем вычислять число Пи по методу Монте-Карло. Возьмём квадрат со стороной $2^31 - 1$, будем генерировать случайные координаты из него нашими генераторами и проверять лежат ли они в четверти круга с таким же радиусом с центром в углу квадрата. Тогда итоговая формула числа Пи

$$\pi \approx \frac{4 \cdot \text{hits}}{\text{iterations}} \quad (4)$$

Эта задача очень хорошо параллелируется, так как генерация чисел и расчёт того подходит оно нам или нет не зависят от других итераций. Только в конце придётся просуммировать количество попаданий в круг с каждого потока.

Для того, чтобы лучше разобраться в том, как работают параллельные вычисления будем писать при помощи pthreads. Это Linux-специфичные потоки, которые под собой содержат создание задач. Их плюс в том, что есть возможность закрепить поток на конкретном ядре, чтобы в замеры не попало время случайной смены контекста, которая бывает очень долгой.

Время закрепления потока на ядре большое по тем же причинам, поэтому будем использовать механизм барьеров, чтобы выполнение вычислений и замер времени начались в одно

время без дополнительных накладных расходов внутри себя. Для этого скажем барьеру, что он должен будет ждать $\text{cores}_{cnt} + 1(+1,,),,,,.,,..$

Для того, чтобы последовательности генератора не пересекались и были уникальным воспользуемся техникой skip-ahead. Каждый поток будет отвечать за свою часть последовательности, длина которой будет равна количеству итераций поделённых на количество создаваемых потоков (потоков будем создавать столько же, сколько ядер, по одному на каждое. Для того, чтобы узнать их количество в runtime можно использовать `sysconf(_SC_NPROCESSORS_ONLN)`). Чтобы преодсчитать сид для каждого потока, нужно возвести в достаточно большую степень наш множитель по модулю (формула была ранее). Для этого будем использовать алгоритм быстрого возведения в степень.

Далее идёт упрощённый код, чтобы показать основную идею реализации.

```

1  uint64_t BASE_MULTIPLIER = 48271ull;
2  struct PiThreadData {
3      size_t iterations = 0;
4      uint32_t seed = 0;
5      size_t hits = 0;
6      pthread_barrier_t* barrier_ptr = nullptr;
7      size_t core_id = 0;
8  };
9
10 static std::vector<PiThreadData> GeneratePiThreadsData(
11     const size_t cores_cnt, const size_t iterations, const uint32_t seed,
12     pthread_barrier_t* const barrier_ptr
13 ) {
14     const uint64_t multiplier = fast_pow_mod(iterations / cores_cnt);
15
16     /*Boring cycle, that create threads_data*/
17     return threads_data;
18 }
19
20 template <GeneratorConcept GenT>
21 static void* PiThreadFoo(void* data) {
22     cpu_set_t cpuset;
23     CPU_SET(data->core_id, &cpuset);
24     pthread_setaffinity_np(pthread_self(), sizeof(cpuset), &cpuset);
25
26     GenT gen(data->seed);

```



```

26
27 pthread_barrier_wait(data->barrier_ptr);
28
29 if constexpr (VectorGenerator<GenT>) {
30     for (size_t iteration = 0; iteration < data->iterations / 8; ++iteration) {
31         const __m512i vec = gen();
32
33         const __m512i x2_vec = _mm512_mul_epu32(vec, vec);
34         const __m512i vec_shifted = _mm512_srli_epi64(vec, 32);
35         const __m512i y2_vec = _mm512_mul_epu32(vec_shifted, vec_shifted);
36         const __m512i dist2_vec = _mm512_add_epi64(x2_vec, y2_vec);
37
38         data->hits += _mm_popcnt_u32(_mm512_cmple_epu64_mask(dist2_vec,
modulus2_vec));
39     }
40 }
41 else if constexpr (ScalarGenerator<GenT>) {
42     for (size_t iteration = 0; iteration < data->iterations; ++iteration) {
43         const uint64_t x = static_cast<uint64_t>(gen());
44         const uint64_t y = static_cast<uint64_t>(gen());
45         data->hits += (x * x + y * y <= MODULUS2);
46     }
47 }
48 return nullptr;
49 }
50
51 template <GeneratorConcept GenT>
52 void SetupPiFunc(
53     const size_t cores_cnt, const size_t iterations, const uint32_t seed,
54     pthread_barrier_t* const barrier_ptr,
55     std::vector<pthread_t>& threads, std::vector<PiThreadData>& threads_data
56 ) {
57     pthread_barrier_init(barrier_ptr, nullptr, cores_cnt + 1);
58     threads_data = GeneratePiThreadsData(cores_cnt, iterations, seed, barrier_ptr);
59     for (size_t thread_num = 0; thread_num < cores_cnt; ++thread_num) {
60         pthread_create(
61             &threads[thread_num], nullptr, PiThreadFoo<GenT>, &threads_data[
thread_num]
62         );

```

```

63     }
64     pthread_barrier_wait(barrier_ptr);
65 }
66
67 size_t DoPiFunc(const size_t cores_cnt, const std::vector<pthread_t>& threads, std::
    vector<PiThreadData>& threads_data) {
68     size_t hits = 0;
69     for (size_t thread_num = 0; thread_num < cores_cnt; ++thread_num) {
70         pthread_join(threads[thread_num], nullptr);
71         hits += threads_data[thread_num].hits;
72     }
73     return hits;
74 }
75
76 template <GeneratorConcept GenT>
77 measurer::Val BenchSomeGen(const size_t cores_cnt, const size_t buckets, const size_t
    batches, const size_t iterations, const uint32_t seed) {
78     /*Create barrier and vectors with threads_data*/
79
80     auto setup_func = [/*...*/]() { SetupPiFunc<GenT>(/*...*/); };
81     auto do_func = [/*...*/]() { return 4. * DoPiFunc(/*...*/) / iterations; };
82
83     const measurer::Val time = measurer::Runner::benchLatency(buckets, batches,
    setup_func, do_func);
84
85     /*destroy barrier*/
86
87     return time;
88 }
89
90 void BenchPi(uint32_t seed, const size_t buckets, const size_t batches, size_t
    iterations) {
91     const size_t cores_cnt = sysconf(_SC_NPROCESSORS_ONLN);
92     const measurer::Val res_scalar = BenchSomeGen<MinstdRand>      (/*...*/);
93     const measurer::Val res_std = BenchSomeGen<std::minstd_rand>    (/*...*/);
94     const measurer::Val res_vec = BenchSomeGen<MinstdRandVec>      (/*...*/);
95
96     /*output*/
97 }

```

Точность вычисления таким методом при 100 000 000 итераций до 4 знака после запятой **3.141504**. Не очень круто, но это издержки метода, он долго сходится.

Реализация	Latency (clks)
std::minstd_rand	269 200 000 $\pm$ 400 000
My MinstdRand	158 600 000 $\pm$ 200 000
My MinstdRandVec	34 400 000 $\pm$ 68 000
Условия замера:	
buckets = 10, batches = 50, iterations = 100 000 000	

Таблица 3: Сравнение производительности на примере вычисления числа Пи